

AIN4312
SPECIAL TOPICS IN
ARTIFICIAL INTELLIGENCE ENGINEERING II

**Controlled AI-Driven Energy Optimization Management System for Smart
Buildings**

ABSTRACT

This is a web-based AI project built to analyze and optimize energy consumption in smart buildings. The system controls a home's electronic devices such as ACs heaters and lights, switching them on and off or adjusting their operating modes to maintain optimal energy consumption without sacrificing comfort. A multi-output gradient boosting model is used to predict indoor temperature and energy consumption, these predictions are then fed into a reinforcement learning model. The RL model uses these outputs in addition to other environmental and user defined variables, to achieve the most satisfactory comfort/energy efficiency balance for the client. The results are later visualized through an interactive website, where the user can observe his/her energy savings, manage multiple homes and devices, and control the system permissions, such as excluding specific devices from automated control. This combination of predictive modeling, reinforcement learning, and user interactive website leads to environmental sustainability and cost efficiency in modern smart buildings.

TABLE OF CONTENTS

ABSTRACT.....	ii
TABLE OF CONTENTS.....	iii
LIST OF TABLES.....	1
LIST OF ABBREVIATIONS.....	2
1. OVERVIEW	3
TEAM	3
1.1. Identification of the need	3
1.2. Definition of the problem.....	4
1.2.1. Functional requirements.....	4
1.2.2. Performance requirements	4
1.3 Conceptual Solution.....	5
1.3.1 Literature Review.....	5
1.4 Architecture.....	5
2. WORK PLAN.....	6
2.1. Work Breakdown Structure (WBS)	6
2.2. Team Responsibilities	7
2.3. Requirement Analysis.....	8
2.3.1 Functional Requirements:	8
2.3.2 Non-Functional Requirements:	8
2.4. Dataset & Preprocessing	9
2.4.1 Devices Configuration Dataset	9
2.4.2 Prediction Model Dataset.....	9
2.4.3 Dataset Preprocessing	10
2.5. Model & System Development.....	10
2.5.1 Model Architecture and Organization:	11
2.6. Testing & Deployment.....	12
2.6.1 Testing Approach.....	12
2.6.2 Handling External Dependencies.....	12
2.6.3 Deployment Strategy.....	12
3. PROGRESS & SUB-SYSTEMS	13
3.3. Devices Management & Configuration	13

3.4. Homes Management & Configuration.....	15
3.5. Consumption and Variables Predication Model	17
3.6. Smart Home Simulator.....	18
3.7. Reinforcement Learning Agent.....	20
3.7.1 Reinforcement Learning Agent Training Loop	21
APPENDIX A1	23
APPENDIX A2	25

LIST OF TABLES

Table 1: List of Abbreviations	2
Table 2: List of Team Members	3
Table 3: Phases Structure	6
Table 4: Work Distribution	7
Table 5: Example of device stored in JSON file	13
Table 6: Example of registered home stored in JSON file.....	15
Table 7: Indoor Environment Dataset (Kaggle Source).....	25
Table 8: Outdoor Weather Dataset (Open-Meteo API)	25
Table 9: UCI Household Energy Consumption Dataset	25

LIST OF FIGURES

Figure 1: Imagine system Architecture	5
Figure 2: Device configuration and action impact	14
Figure 3: Home setting and action space	16
Figure 4: Position of the Prediction Model within the System Architecture	17
Figure 5: Summary of SmartHomeEnv class	18
Figure 6: Trainable DQN layer for RL.....	20
Figure 7: Model Prediction Accuracy (Actual vs Predicted)	26

LIST OF ABBREVIATIONS

Table 1: List of Abbreviations

Abbreviation	Full Form	Meaning
kWh	Kilowatt-hour	Unit of energy: 1 kW used for 1 hour = 1 kWh
kW	Kilowatt	Unit of power: rate of energy use or generation
Wh	Watt-hour	Smaller unit of energy (1 kWh = 1,000 Wh)
NNs	Neural Networks	Computational models inspired by the brain, used for pattern recognition and learning from data
RL	Reinforcement Learning	A learning paradigm where agents learn by interacting with an environment and receiving rewards
DQN	Deep Q-Network	A type of deep reinforcement learning algorithm that uses neural networks to approximate Q-values for decision-making
UI	User Interface	The visual and interactive elements users engage with (buttons, menus, screens)

1. OVERVIEW

The system aims to build user friendly website that allows its users to easily analyze the system's performance and manage their homes and devices through operations such as add, delete, rename, set permissions and many more. Each home will have a separate list of devices and permissions to guide our agent. The RL agent will be trained on each home separately to ensure optimization tailored to the specific characteristics and requirements of that environment. The agent will be trained first on simulations made by our prediction model to mitigate the potential risks associated with deploying an untrained RL agent.

TEAM

Our team consists of five AI department students each one will contribute evenly to ensure high integration and fair work experience for all.

Student Name	Student Number
Aws Firas Harith	2268408
Alhasan Alobaidi	2267156
Abdulrahman Alamoudi	2104927
Hashem Khurd	2267527
Sulaiman Alareer	2106738

Table 2: List of Team Members

1.1. Identification of the need

Traditional smart home systems often rely on fixed schedules or manual user input, which can lead to inefficient energy consumption and unnecessary costs. These inefficiencies lead to environmental harm and financial burdens, especially in today's economical environment. The proposed system utilizes AI models and agents to monitor energy usage patterns in real time, dynamically adjusting devices to minimize energy waste and reduce costs. By integrating reinforcement learning, the system continuously learns the right decisions to optimize energy consumption while maintaining occupant comfort.

1.2. Definition of the problem

The challenge lies in maintaining an optimal indoor environment while minimizing energy waste. Traditional systems depend heavily on manual adjustments which are more prone to errors because of human clumsiness and the lack of predictive intelligence to adapt to changing conditions. To address this, the proposed system employs a gradient boosting model to forecast short-term temperature and energy consumption trends, providing the reinforcement learning (RL) agent with predictive insights. The RL model then uses this information to autonomously manage device operations ensuring occupant comfort while optimizing energy consumption dynamically and efficiently.

1.2.1. Functional requirements

List of the system's main functions:

- **Devices management:** This will handle all devices' action (add devices, remove devices, assign permission, set the maximum kWh consumption).
- **Home management:** This will handle all possible home actions e.g. (add home, delete home, rename home).
- **Home reading:** This function collects data from sensors for future predictions.
- **Home simulator:** this function will mimic a home environment if the available data is not enough.
- **RL agent:** This function will be responsible for creating the DQN and initializing the weights and setting the parameters for convergence.
- **Web Interface:** A user friendly website that visualizes the performance and manages the system.

1.2.2. Performance requirements

The prediction of the system should follow a specific policy, which will directly affect the total rewards returned to the RL agent. We will tackle the policy function to ensure that RL model never takes actions that increase the reward total in favor of overall comfort rate in the house.

- The prediction model should achieve a high level of accuracy (0.8 or higher).
- The RL system should achieve the best comfort / efficiency balance.
- Model response time must remain under one second to ensure real-time adaptability.
- The website should be accessible and user friendly.

1.3 Conceptual Solution

The system combines a predictive layer (based on XGBoost regression) that estimates hourly energy usage and temperature, and a decision layer (based on Deep Q-Network reinforcement learning) that chooses the optimal device actions. The learning loop continuously improves decision accuracy based on the reward/penalty policy. To display the results of our system we need to build a web based central management system with homes and devices for our models to optimize and a dashboard to display and visualize the results.

1.3.1 Literature Review

Studies in intelligent energy management highlight reinforcement learning as an effective optimization approach. Research published in IEEE (2023) demonstrated that DQN based controllers reduced HVAC energy consumption by up to 25%. Similarly, Elsevier (2024) reported that AI-driven predictive systems maintained stable indoor comfort while cutting costs significantly. MIT and Google DeepMind experiments on adaptive energy control validated the use of RL for minimizing peak loads while ensuring system stability. These findings support the integration of XGBoost for forecasting and reinforcement learning for dynamic control in this project.

1.4 Architecture

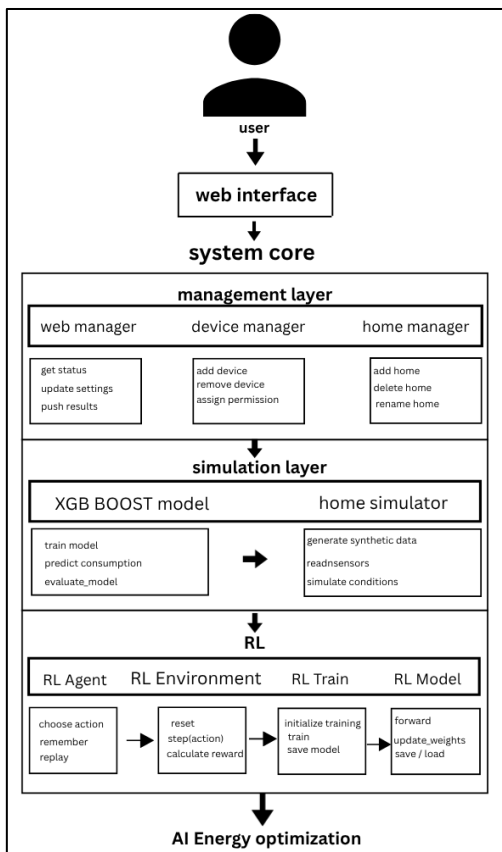


Figure 1: Imagine system Architecture

1. Homes and Devices management – impact calculation: Manages metadata and real-time status of homes, appliances, and sensors.
2. Prediction layer: Forecasts energy demand, device usage, or environmental variables.
3. RL initializing and training: Initializes reinforcement learning agents with environment parameters.
4. Logging the training outputs: Captures metrics, rewards, policy evolution, and agent decisions.
5. RL testing layer – simulator: Evaluates trained agents in virtual environments.
6. Web visualizing: Enables user-friendly exploration of predictions, RL behavior, and device impact.

2. WORK PLAN

2.1. Work Breakdown Structure (WBS)

The project was divided into 6 main stages, and it is illustrated below.

Phase No.	Duration in weeks	Key Activities
1	1-3	Structuring, Data collection
2	4-5	Data cleaning, Prediction model training
3	6-7	Home & Devices Mang., log system
4	8-10	RL agent, RL training, RL testing
5	11-12	APIS, code cleaning
6	13-14	Web Manag. site

Table 3: Phases Structure

2.2. Team Responsibilities

Phase No.	Responsibilities
1	AWS: Project Structuring. ALHASSAN: Project Structuring. ABDULRAHMAN: Related Work searching. HASHEM: Data Collection. SULAIMAN: Data Collection.
2	AWS, ALHASSAN, ABDULRAHMAN: Same phase 1 with documentation. HASHEM: Model Training. SULAIMAN: Data Cleaning & Merging.
3	AWS: Building Device Manag. ALHASSAN: Building Homes Manag. ABDULRAHMAN: Organize the files and fixing debugs and implement structure. HASHEM: Test the functions integrations. SULAIMAN: Implement the log files for next training.
4	AWS: Test the home environment simulator. ALHASSAN: Search for simple RL agents. ABDULRAHMAN: Build the RL agent. HASHEM: Build the train loop and save the result using log structure. SULAIMAN: Build a simulator to test the RL performance.
5	AWS: Code cleaning. ALHASSAN, HASHEM: Documenting the previous works ABDULRAHMAN, SULAIMAN: Build the APIs & testing.
6	ALHASSAN: Building the Web. ABDULRAHMAN: API & Web integration. HASHEM: Testing the web edge cases. AWS, SULAIMAN: Documenting.

Table 4: Work Distribution

2.3. Requirement Analysis

The goal of this requirement analysis is to clarify what the system needs as general in order to function effectively as an intelligent home energy management platform. This subsection focuses on the fundamental capabilities required to observe the environment, understand energy behavior, and prepare the ground for intelligent control.

2.3.1 Functional Requirements:

The system must continuously represent the indoor thermal state of the environment. Indoor temperature is the primary indicator of user comfort and directly affects the operation of climate-related devices such as air conditioners and heaters. To support this, the system maintains indoor temperature as a core variable, bounded by a configurable comfort range for each home. This allows the system to reason about comfort in a consistent and measurable way. Moreover, The system must be able to measure and accumulate energy usage during operation. Energy consumption is central to the problem being addressed, as the objective is to reduce unnecessary energy use while maintaining acceptable comfort levels. By tracking cumulative energy consumption in kilowatt-hours, the system can clearly quantify the cost of each device action and evaluate how control decisions affect overall efficiency. In addition to that, The system should be able to estimate near-future indoor temperature and energy consumption based on recent behavior. Predictive insight is important because many home control decisions have delayed effects. A short-horizon prediction model is therefore used to forecast upcoming temperature and energy trends. The close match between predicted and actual values demonstrates that the system can anticipate environmental changes with sufficient accuracy to support smarter decision-making in later stages.

2.3.2 Non-Functional Requirements:

User-Friendly Interaction and System Usability:

The system should provide a user-friendly and intuitive interface that allows users to interact with the smart home environment without requiring technical expertise. All core functionalities, such as monitoring home status, managing devices, and viewing system performance, should be easily accessible and clearly presented. The design should prioritize clarity and simplicity to support both technical and non-technical users.

Reproducibility and Controlled Training Management:

The system should support reproducible training by storing models, configurations, and training parameters in a structured and consistent manner. At the same time, it should allow controlled variation in training conditions to improve generalization.

2.4. Dataset & Preprocessing

Two distinct datasets are used in this system, each serving a different but complementary purpose. One dataset is responsible for device representation and action modeling, while the second dataset is used for training the prediction model that estimates indoor temperature and energy consumption and other variables. Separating these datasets allows the system to remain modular, scalable, and easier to extend.

2.4.1 Devices Configuration Dataset

The first dataset describes the controllable devices within the smart home environment. This dataset defines what devices exist, how much energy they consume in baseline operation, and which actions are allowed for each device.

Each device entry includes:

- A device name
- A base energy consumption value (kWh)
- A list of supported control actions (permissions)

This dataset is stored in a structured JSON format and is loaded dynamically at runtime. It is not used for numerical model training, but instead serves as a configuration source that drives:

- Action space generation
- Energy cost calculation
- Consistency between device behavior and environment simulation

To support flexible system evolution, this dataset is designed to be editable. When devices or permissions are updated, the system automatically recalibrates the action impact parameters so that energy and temperature effects remain consistent. This ensures that changes to device definitions do not require manual updates to the learning or simulation logic.

2.4.2 Prediction Model Dataset

The second dataset is dedicated to training the prediction model used to estimate short-term indoor temperature and cumulative energy consumption. This dataset is structured as a **time-series**, where each data record represents the system state at a specific time step.

The dataset primarily includes:

- Historical indoor temperature values
- Corresponding energy consumption values
- Temporal ordering information to preserve sequence continuity

This dataset is used to train a supervised XGBOOST prediction model that learns how temperature and energy evolve over time based on previous states.

2.4.3 Dataset Preprocessing

Both datasets undergo preprocessing steps tailored to their roles, but guided by common principles of consistency, validation, and stability. First, data validation and cleaning are applied. For the device configuration dataset, this involves verifying the integrity of device definitions, ensuring that energy values are valid and that action names follow consistent naming conventions. For the prediction dataset, numerical validity checks are applied to eliminate missing, corrupted, or ill-formed values that could disrupt model training.

Second, normalization and standardization are performed where applicable. While device configuration data remains largely categorical and structured, numerical values such as baseline energy consumption are normalized conceptually to ensure comparable scaling across devices. In the prediction dataset, continuous variables such as indoor temperature and energy consumption are explicitly scaled to stabilize learning and improve convergence.

Third, structural alignment is enforced. Device configuration data is transformed into a unified internal representation that enables automatic generation of valid control actions. In parallel, the prediction dataset is reorganized into fixed-length temporal sequences, preserving the chronological order required for time-series learning while ensuring compatibility with the prediction model interface. Finally, consistency across subsystems is maintained through preprocessing. Changes in device definitions automatically propagate to energy and temperature impact modeling, ensuring that simulated behavior aligns with device configurations. Similarly, the processed prediction dataset produces outputs that are compatible with the system state representation, allowing predicted values to be integrated seamlessly when required.

2.5. Model & System Development

The overall system is designed as a collection of cooperating subsystems, where each subsystem has a clearly defined responsibility and operates independently from the others. This modular approach simplifies development, improves maintainability, and allows the system to scale as new features and learning components are introduced.

Each subsystem in the system serves a single, focused objective:

The **device management subsystem** is responsible for handling all device-related functionality. It manages the available devices, their baseline energy consumption, and their supported operational modes. This ensures that device behavior is consistently represented across the system and that only valid actions can be executed.

The **home management subsystem** is responsible for organizing homes and their internal structure. It allows homes to be created, removed, and managed independently. Each home maintains its own comfort preferences and associated devices, enabling personalization without affecting other homes.

The **prediction subsystem** is dedicated exclusively to forecasting short-term trends in indoor temperature and energy consumption. It does not issue control actions or manage devices; instead, it provides estimated future states that can be optionally used by other components. This separation ensures that predictive logic remains isolated and reusable.

The **reinforcement learning agent subsystem** is responsible for decision-making. It learns a control policy based on the observed system state and reward feedback. The agent itself does not interact directly with devices or external data sources; all interactions occur through the environment abstraction.

The **home environment subsystem** acts as the central integration layer. It is responsible for collecting inputs from external sources such as APIs or IoT sensors, maintaining the current state of the home, applying device actions, and exposing observations and rewards to the learning agent. This design ensures a clean separation between real-world data acquisition and learning logic.

2.5.1 Model Architecture and Organization:

Two main types of models are implemented within the system.

The first model type is the prediction model, which is trained to estimate near-future indoor temperature and energy consumption based on historical data. This model supports anticipatory reasoning by capturing short-term trends in system behavior. The prediction model is independent of home-specific identities and can be retrained or replaced without affecting the learning agent.

The second model type is the reinforcement learning model, which learns a control policy for managing energy usage while maintaining comfort. Unlike the prediction model, this model is home-specific. Each home has its own trained reinforcement learning model, saved and referenced using the home's name. This ensures that learning remains personalized and that no interaction or interference occurs between different homes. Each home-specific model is trained using its own environment configuration, comfort thresholds, and device set. Optimizers and training parameters are standardized across homes to maintain consistency, while the learned policies remain distinct. This design allows users to retrain, refine, or reuse models for individual homes independently.

2.6. Testing & Deployment

Testing and deployment are handled as an integral part of system development to ensure reliability, consistency, and robustness. The system is designed to operate correctly even when external dependencies such as APIs or IoT devices are partially unavailable or return unexpected values.

2.6.1 Testing Approach

Each subsystem is tested independently immediately after implementation to verify that it performs its intended function correctly. This includes validating inputs, outputs, and internal states. If a function encounters invalid data or an unavailable external service, it is required to raise a controlled error or apply a safe fallback behavior without causing the system to crash.

Before integration, individual components are tested in isolation. These tests confirm that each subsystem produces consistent and predictable outputs when provided with valid inputs. This stage ensures that device management, home management, prediction logic, environment updates, and learning components function correctly on their own.

After integration, the full system is tested as a combined workflow. These tests focus on verifying that data flows correctly between subsystems and that state transitions remain consistent throughout execution. Particular attention is given to ensuring that interactions between components do not introduce unintended side effects or mismatched input–output formats.

2.6.2 Handling External Dependencies

The system accounts for the fact that external data sources such as APIs and IoT devices may occasionally fail, return incomplete data, or be temporarily unavailable. In such cases, the system is designed to handle errors gracefully. Instead of terminating execution, components report the error, log relevant information, and continue running using the last known valid state or default safe values. This approach ensures continuous operation and prevents system instability caused by external failures.

2.6.3 Deployment Strategy

Deployment is incremental and modular. Subsystems can be deployed and updated independently, reducing the risk of system-wide failures. The system supports both simulated and real-data operation, allowing thorough validation before deployment in real environments. Trained models are saved and managed at the home level, enabling selective loading, retraining, and reuse without interfering with other homes or system components.

3. PROGRESS & SUB-SYSTEMS

This section presents the system components that have been designed and implemented up to the current stage of development. Each subsection describes a specific subsystem that contributes to the overall functionality of the smart home platform. The subsections included here reflect the parts of the system that are already operational and integrated, forming a stable foundation for the remaining components to be extended in later stages.

The following subsections describe the implemented subsystems in a progressive manner, starting from configuration-level components and moving toward learning-based decision-making.

3.1. Devices Management & Configuration

Device Name	Base kWh	Permission
Air Conditioning	3.5	e.g, ["turn_on", "set_low", ...]

Table 5: Example of device stored in JSON file

The Devices Management & Configuration subsystem is responsible for defining, maintaining, and validating all smart devices available within the system. This subsystem provides a centralized and consistent way to describe device behavior, energy characteristics, and supported control actions, which are later used by both the simulation environment and the learning agent. The core strategy of this subsystem is to separate **device definition** from **device execution logic**. Devices are treated as configurable entities rather than hardcoded objects. This allows devices to be added, updated, or removed without modifying the learning or simulation code.

Device Catalog Management:

All device definitions are stored in a persistent device catalog using a structured JSON format. This catalog serves as the single source of truth for all device-related information in the system. An example of a device entry stored in the catalog is shown in **Table 5**.

Table 5 illustrates how each device is represented using a unique name, a baseline energy consumption value (kWh), and a list of permitted actions. The catalog is loaded at system startup through the device management component, and all subsequent operations reference this validated in-memory representation. Any modification to the catalog is saved persistently to ensure consistency across system runs.

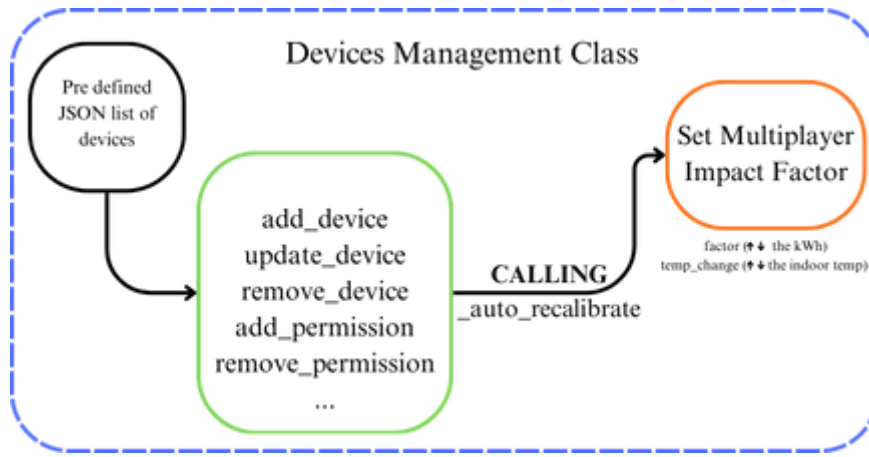


Figure 2: Device configuration and action impact

Permissions and Action Definition:

Each device exposes a predefined set of permissions representing allowable actions, such as turning the device on or off, switching operating modes, or selecting energy-saving configurations. Permissions are defined as descriptive keywords rather than numeric values, improving readability and reducing ambiguity. These permissions later serve as the basis for constructing the system's action space. By constraining actions strictly to declared permissions, the system prevents invalid or undefined control behavior.

Impact Calibration and Configuration Consistency:

To ensure that device actions have measurable and consistent effects, device permissions are automatically linked to an impact configuration. When device definitions or permissions change, an impact calibration process is triggered to generate or update the corresponding parameters, such as energy multipliers and temperature influence values. This relationship between device configuration and action impact is conceptually illustrated in **Figure 2.**, which shows the flow from the device catalog to validated actions and their effects within the system. By recalibrating impacts automatically, the system maintains alignment between configuration data and simulated physical behavior without requiring manual tuning.

3.2. Homes Management & Configuration

Default					
comfort_range		rooms			
min	max	room name	Living Room	Bedroom_1	Kitchen
22	24	devices	["Air Conditioning", ...]	["Heater", "Lights", ...]	["Oven", "Dishwasher", ...]

Table 6: Example of registered home stored in JSON file

The Homes Management & Configuration subsystem is responsible for organizing the system around user-defined homes. It defines how devices are grouped, how comfort preferences are set, and how each home is treated as an isolated and independent control unit. This subsystem enables personalization while maintaining a shared system infrastructure.

Home Representation and Structure:

Each home is defined using a structured configuration that includes:

- A unique home name
- A comfort temperature range (minimum and maximum)
- A set of rooms

This structure reflects a realistic residential layout and allows devices to be grouped logically rather than managed as a flat list. An example of a registered home configuration stored in the system is shown in **Table 6**. As illustrated, devices are grouped by rooms, reflecting a realistic household layout and enabling logical organization rather than flat device management.

Home Configuration Storage:

Home configurations are persistently stored in a dedicated configuration file using a structured JSON format. This file is loaded at system startup and serves as the authoritative source for all home-related settings.

The subsystem supports the following operations:

- Adding new homes
- Removing existing homes
- Updating comfort ranges
- Assigning or removing devices from rooms

All changes are immediately saved, ensuring that home configurations are preserved across sessions and system restarts.

Device Scoping and Action Space Reduction:

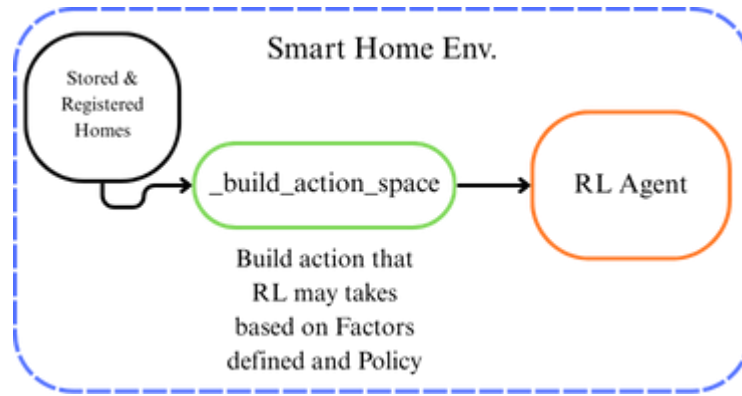


Figure 3: Home setting and action space

When a home is selected for simulation or training, only the devices explicitly associated with that home are activated within the system environment. This scoping mechanism is illustrated conceptually in Figure 3, which shows how home configuration limits the devices exposed to downstream subsystems.

- By restricting the active device set to those relevant to the selected home:
- The overall action space is significantly reduced
- Decision-making becomes more efficient
- Learning stability is improved by eliminating irrelevant actions

This design ensures that control policies remain tightly aligned with the physical environment they are intended to optimize.

3.3. Consumption and Variables Predication Model

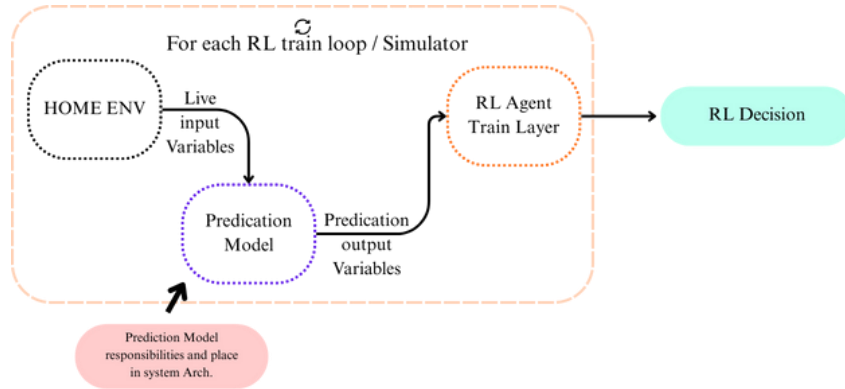


Figure 4: Position of the Prediction Model within the System Architecture

The Consumption and Variables Prediction Model is responsible for estimating short-term trends in indoor temperature and energy consumption based on historical observations and environmental context. This model acts as a forecasting layer that enhances the system's understanding of how the home environment evolves over time, without directly controlling devices or enforcing actions. As illustrated in **Figure 4**, the prediction model is embedded within the system architecture as a supporting component. It operates independently from device execution and reinforcement learning logic, allowing it to be developed, evaluated, and refined without impacting other control subsystems.

Model Objective and Scope:

The primary objective of this model is to predict:

- Future indoor temperature values
- Future energy consumption levels

These predictions are intended to provide contextual awareness rather than direct decision-making. By anticipating system behavior, the model supports more informed control strategies in later stages of the system while remaining decoupled from the reinforcement learning agent.

Input Variables and Data Preparation:

The prediction model relies on a combination of internal and external variables that influence indoor conditions and energy usage. These variables include historical indoor environmental measurements, external weather conditions, and household energy consumption signals. All datasets used for training and evaluation undergo preprocessing to ensure temporal alignment and numerical consistency. This includes cleaning missing values, normalizing time formats, and merging multiple data sources into a unified time-series representation. A detailed description of the datasets, preprocessing pipeline, and data sources is provided in **Appendix A2**.

Model Outputs and System Integration:

The prediction model produces numerical forecasts for indoor temperature and energy consumption over a short horizon. These outputs are structured to match the internal state representation of the smart home environment. When enabled, the predicted values can be incorporated as additional state variables for higher-level subsystems, such as the smart home simulator or reinforcement learning agent. However, the prediction model itself remains isolated from control logic, ensuring modularity and flexibility in system design.

3.4. Smart Home Simulator

The Smart Home Simulator represents the core integration layer of the system. It is responsible for bringing together device configurations, home settings, external data sources (APIs and IoT devices), and system dynamics into a unified environment that can be consumed by higher-level learning and control components. This subsystem acts as the execution context where system state is updated, actions are applied, and outcomes are evaluated in a consistent and controlled manner.

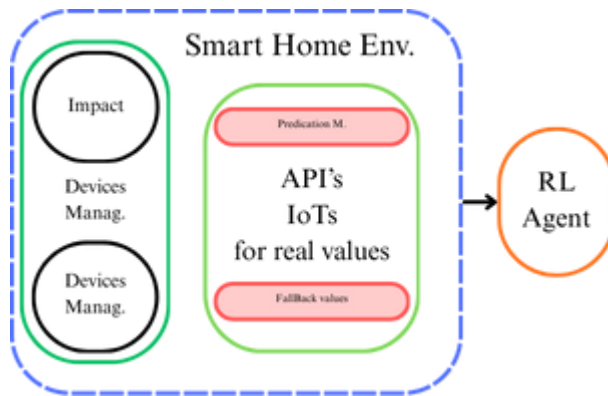


Figure 5: Summary of SmartHomeEnv class

Role and Design Overview:

The simulator is implemented as an environment abstraction that encapsulates the full operational state of a selected home. As illustrated in **Figure 5**, it serves as the central interaction point between configuration subsystems (devices and homes), external data sources, and the reinforcement learning agent. Its design follows a modular strategy where configuration, sensing, simulation, and evaluation responsibilities are clearly separated but coordinated through a unified interface.

Environment Initialization and Configuration Loading:

At initialization, the simulator loads all required configuration data:

- Home-specific settings, including comfort temperature range and assigned devices
- Global device definitions from the device catalog
- Impact rules that map device actions to energy and temperature effects

When a specific home is selected, only devices assigned to that home are activated within the simulator. This ensures that the simulated environment accurately reflects the physical structure of the home being modeled and remains consistent with the configuration described in Sections 3.1 and 3.2.

Integration with APIs and IoT Devices:

The simulator is designed to operate in two modes:

- **Real-data mode**, where inputs are retrieved from external APIs or IoT sensors
- **Simulation mode**, where values are generated internally for testing and training

External inputs include outdoor temperature, indoor temperature, and real energy consumption when available. If an API or IoT device fails to respond or returns invalid data, the simulator does not terminate execution. Instead, each input function handles errors gracefully by reporting the issue and continuing operation using safe fallback values.

This approach ensures robustness and uninterrupted execution, which is essential when dealing with real-world data sources that may be unreliable.

Environment Dynamics, Rewards, and Episode Control:

The simulator advances in discrete time steps. At each step, it applies the selected action, updates the environment state, and computes a reward value.

The reward function balances two competing objectives:

- Minimizing energy consumption
- Maintaining indoor temperature within the defined comfort range

Each simulation episode represents a full operational cycle (e.g., one day), after which the environment is reset. This structure allows repeated experimentation and consistent evaluation of learning behavior.

The Smart Home Simulator serves as the operational backbone of the system. By integrating configuration data, external inputs, realistic dynamics, and reward evaluation in a single environment abstraction, it enables safe, consistent, and scalable experimentation with intelligent control strategies. Its modular and fault-tolerant design ensures reliable operation in both simulated and real-data settings, the fallback of APIs explained in **Appendix A1**.

3.5. Reinforcement Learning Agent

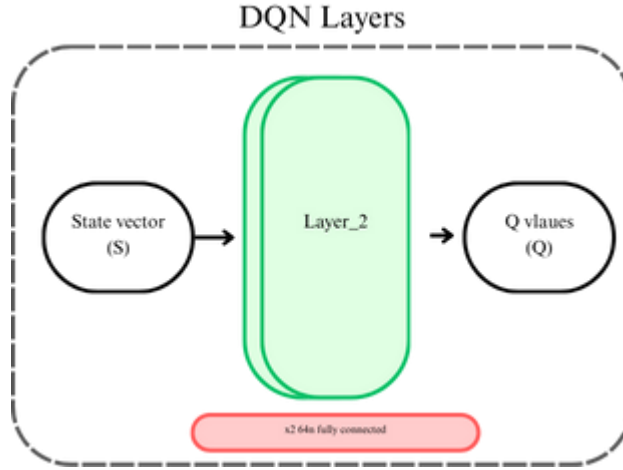


Figure 6: Trainable DQN layer for RL

The Reinforcement Learning (RL) Agent is responsible for learning optimal control policies that balance energy efficiency and indoor comfort within the smart home environment. This agent operates using a **Deep Q-Learning (DQN)** approach, enabling it to approximate action-value functions over a continuous state space and a discrete action space. As shown conceptually in **Figure 6**, the agent relies on a neural network to estimate Q-values for all possible actions and uses these estimates to guide decision-making during interaction with the smart home simulator.

Deep Q-Network Architecture

The core of the RL agent is a feed-forward deep neural network that approximates the Q-function $Q(s, a)$. The network takes the current environment state as input and outputs a Q-value for each valid action.

The architecture consists of:

- An input layer matching the environment state dimension
- Two fully connected hidden layers with 64 neurons each
- ReLU activation functions between layers
- A linear output layer with dimension equal to the action space size

This structure allows the network to capture non-linear relationships between indoor conditions, energy consumption, and action effectiveness while remaining computationally efficient for training. The neural network definition is implemented using PyTorch and forms the function approximator of the DQN algorithm

Model Persistence and Reuse

To support continuity and personalization, the RL agent provides functionality to save and load trained models. Trained network weights are persistently stored using a standardized file structure. When a previously trained model is available, the agent can load it and continue operation without retraining from scratch. If a stored model is incompatible with the current network architecture, the system automatically reinitializes the network using safe weight initialization. This ensures robustness during system evolution while preserving compatibility with existing trained models. This design allows users to reuse trained models across sessions, retrain models incrementally, and maintain home-specific learning behavior without interference between households.

Integration with the Smart Home Simulator

During execution, the RL agent interacts directly with the smart home simulator in a closed training loop. At each step, the agent:

1. Observes the current environment state
2. Selects an action based on its policy
3. Receives a reward and next state from the simulator
4. Stores the experience in memory
5. Updates network weights through replay when sufficient data is available

This interaction enables the agent to gradually learn control strategies that minimize energy usage while maintaining comfort constraints.

3.5.1 Reinforcement Learning Agent Training Loop

The reinforcement learning agent is trained through an episodic interaction loop with the smart home simulator. The training loop coordinates environment initialization, action selection, experience collection, network optimization, and performance logging, enabling the agent to iteratively improve its control policy over time. If a previously trained model exists for the selected home, the agent attempts to load it before training begins. This allows training to resume from an earlier state rather than starting from random initialization. If no compatible model is found, training proceeds from scratch using randomized weights.

Episode-Based Training Structure

Training is organized into episodes, where each episode represents a full operational cycle of the smart home (e.g., one day). At the beginning of each episode, the environment is reset and the initial system state is obtained.

Within each episode, the agent repeatedly:

1. Observes the current environment state
2. Selects an action using an ϵ -greedy policy
3. Executes the action in the simulator
4. Receives a reward and the next state
5. Stores the experience for later learning

This interaction continues until a terminal condition is reached, such as the completion of a full simulation cycle.

Performance Monitoring and Logging

Throughout training, key performance indicators (KPIs) are recorded after each episode. These include total reward, total energy consumption, average indoor temperature, comfort violations, exploration rate, and training loss. KPI values are logged persistently and used to generate summary plots for performance analysis. This logging mechanism allows quantitative evaluation of training stability, convergence behavior, and policy improvement over time.

Model Checkpointing and Finalization

To preserve training progress, model checkpoints are saved periodically at predefined episode intervals. At the end of training, a final model snapshot is stored for deployment or further refinement. All saved models are associated with the specific home configuration used during training, ensuring isolation and reproducibility across homes.

APPENDIX A1

This appendix provides an in-depth view of the critical components of the Smart Home Environment that enable integration between configuration data, external APIs, IoT inputs, and reinforcement learning. The purpose of this appendix is to document implementation-level decisions that are essential for robustness and functionality but are too detailed for the main body of the report.

Environment Initialization and Home Context Binding

At initialization, the environment dynamically binds to a specific home configuration and loads only the devices associated with that home. This ensures full isolation between homes and prevents cross-contamination during training or simulation.

```
if self.home_name and self.home_name in self.home_manager.homes:
    self.devices = {
        d: self.manager.get_all_devices()[d]
        for d in self.home_manager.get_home_devices(self.home_name)
    }
    self.comfort_min, self.comfort_max =
self.home_manager.homes[self.home_name].get("comfort_range", comfort_range)
```

External APIs and IoT Integration with Fault Tolerance

The environment supports real-data operation by retrieving inputs from external APIs and sensors. These include:

- User geolocation
- Outdoor temperature
- Indoor temperature (IoT)
- Energy consumption (IoT)
- Working day / weekend detection

Each input source is wrapped in exception handling to prevent system crashes when external services fail e.g, below for an outdoor temperature API.

```
def _out_temp(self):
    try:
        self.outdoor_temp = get_real_outdoor_temp(self.lat, self.lon) # the
error will raised from api
    except Exception as e:
        self.outdoor_temp = random.uniform(10, 40)
```

Reward Function Implementation

The reward function enforces a trade-off between comfort and energy efficiency.

```
reward = -(energy_used * energy_weight + comfort_penalty * 1.90) + comfort_reward)
```

Reward characteristics:

- Penalizes excessive energy usage
- Penalizes deviation from comfort range
- Rewards stable operation within comfort bounds
- Uses dynamic weighting to discourage high-load actions

This formulation aligns with the objectives defined in Section 3.6 and provides stable gradients for reinforcement learning.

Episode Control and Reset Logic

Each episode corresponds to a full daily simulation cycle.

```
done = self.step_count >= 24
```

Reset logic restores the environment to a clean initial state while reloading external data:

```
def reset(self):  
    self._out_temp()  
    self._indoor_temp()  
    self._real_kWh()  
    self._is_weekend()
```

This guarantees repeatable training cycles and reproducible evaluation.

This appendix highlights the critical implementation aspects of the Smart Home Environment, which serves as the execution layer integrating home configurations, device definitions, external APIs, and reinforcement learning interaction. The environment initializes in a home-scoped manner, activating only devices assigned to the selected home and applying home-specific comfort constraints. This design ensures strict isolation between homes, prevents invalid actions, and enables consistent reuse of trained policies. To support real-world operation, the environment integrates external APIs and IoT sensors for weather, indoor conditions, energy consumption, and working-day context, with fault-tolerant handling that allows safe fallback behavior when data sources are unavailable. The environment dynamically constructs its action space based on configured devices and permissions, ensuring safe and configuration-driven control. State transitions combine device-specific energy usage, calibrated temperature effects, and passive heat exchange with the outdoor environment to balance realism and computational efficiency. The reward function penalizes excessive energy consumption and comfort violations while rewarding stable operation within defined comfort bounds. Episodic control and reset mechanisms ensure repeatable training cycles and reproducible evaluation. Collectively, these components make the Smart Home Environment a robust, scalable, and reliable.

APPENDIX A2

We started with the predictive model by collecting, cleaning, and merging 3 datasets. We dropped empty rows and forward filled some empty values and formulated the datasets into hourly format and matched the timestamps for accurate merging.

Timestamp	Temp (C)	Humidity (%)	CO (ppm)	HVAC Temp (C)	Energy (kWh)
2024-02-05 08:00	21.87	37.7	407	21.4	1.32
2024-02-05 08:15	24.75	43.1	430	24.7	0.90
2024-02-05 08:30	23.65	48.1	426	24.2	0.72
2024-02-05 08:45	22.99	45.9	365	23.6	1.01
2024-02-05 09:00	20.78	47.0	364	20.3	1.04

Table 7: Indoor Environment Dataset (Kaggle Source)

Explanation: This dataset includes records of temperature, humidity, and CO concentration readings, essential for simulating internal building behavior.

Time	Temperature (C)	Humidity (%)	Wind Speed (m/s)
2024-01-01T00:00	9.9	82	9.2
2024-01-01T01:00	8.7	90	6.1
2024-01-01T02:00	8.4	89	5.9
2024-01-01T03:00	8.6	87	7.2
2024-01-01T04:00	8.9	86	7.7

Table 8: Outdoor Weather Dataset (Open-Meteo API)

Explanation: The external environment dataset tracks weather patterns that influence indoor temperature and system performance.

Datetime	Global Active Power (kW)	Voltage (V)	Global Intensity (A)	Sub 1	Sub 2
2006-12-16 17:24:00	4.216	234.8	18.4	0	1
2006-12-16 17:25:00	5.360	233.6	23.0	0	1
2006-12-16 17:26:00	5.374	233.2	23.0	0	2
2006-12-16 17:27:00	5.388	233.7	23.0	0	1
2006-12-16 17:28:00	3.666	235.6	15.8	0	1

Table 9: UCI Household Energy Consumption Dataset

Explanation: Provides detailed measurements of household power usage and serves as the baseline for performance comparison.

We tried different approaches for predicting indoor temp and energy consumption, such as LSTMs and random forest and found that gradient boosting gave us the best results (shown below).

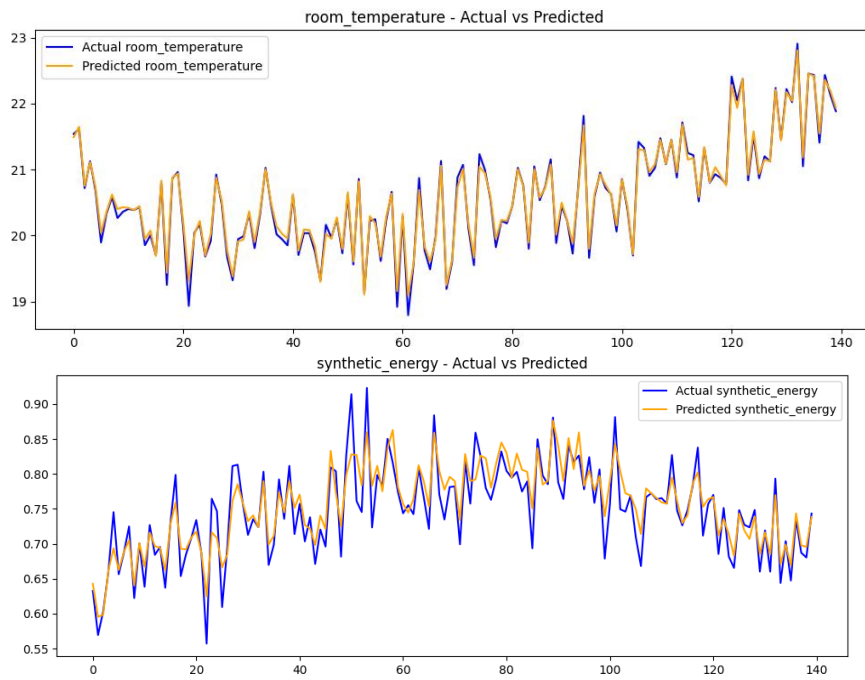


Figure 7: Model Prediction Accuracy (Actual vs Predicted)

Explanation: The XGBoost multi-output regression achieved near-perfect prediction for temperature and high precision for energy consumption, confirming its suitability for real-time forecasting.